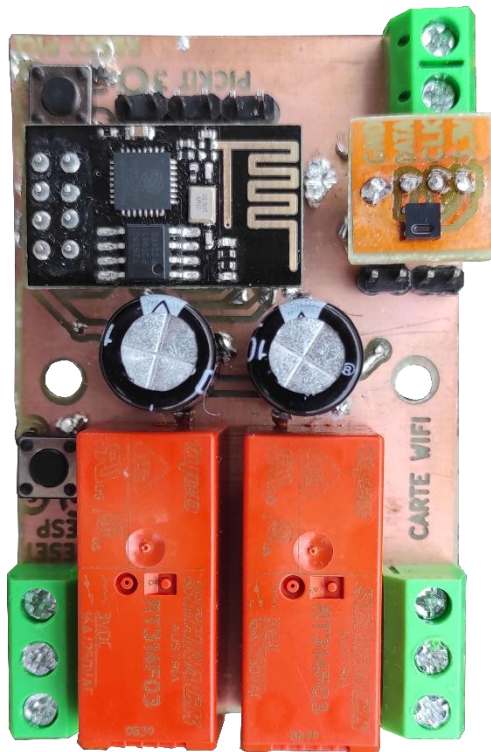


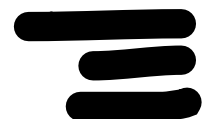
Université Grenoble Alpes
Institut universitaire de technologie 1
Département Génie Électrique et Informatique Industrielle

Rapport technique : Télécommande Wi-Fi



Raphael BARRAUD & Mathis ROJAS
2^{ème} Année ESE
Promotion 2023-2026

Enseignants :



1.	Présentation du projet	3
1.1.	Introduction	3
1.2.	Cahier des charges	3
1.3.	Gestion de projet	4
2.	Etat de l'art	5
3.	Présentation de l'ESP01	5
4.	Prise en main de la console et de l'ESP	6
4.1.	La Chamberod-console pour linux	6
4.2.	Configuration de l'ESP et commande AT	6
5.	Etablissement d'une communication	7
5.1.	UDP/IP.....	7
5.2.	TCP/IP.....	8
6.	Choix des composants principaux	8
6.1.	Capteurs de température.....	8
6.2.	Relais.....	9
6.3.	Régulateur de tension.....	10
7.	Développement des 2 cartes	11
7.1.	Carte capteur	11
7.2.	2 ^{ème} Carte wifi	11
8.	Programmation de la Carte WIFI	14
8.1.	Présentation globale du programme.....	14
8.2.	Gestion des interruptions	14
9.	Télécommande wifi (Application PC).....	15
10.	Consommation électrique	16
11.	Conclusion.....	17
12.	Bibliographie	17
12.1.	Datasheets des composants	17
12.2.	Ressources utilisées pour le développement et la programmation du projet	17
13.	Annexes	18
13.1.	Schéma électrique et PCB de la 1 ^{ère} Carte Wifi	19
13.2.	Communication UART entre l'ESP et le PIC18F26K22	20
13.3.	Programme C de la carte WIFI (PIC18F26K22).....	20
13.4.	Codes node.js, HTML et CSS	25

1. Présentation du projet

1.1. Introduction

Notre projet consiste en la conception et la réalisation d'une carte Wi-Fi autonome en énergie destinée à des applications de domotiques. La carte sera équipée d'un module Wi-Fi ESP01 (basé sur la technologie ESP8266), de capteurs permettant de mesurer la température et l'humidité, ainsi que de relais pour piloter divers actionneurs.

Le système sera piloté par un microcontrôleur PIC18F26K22. Cette carte devra être contrôlable sans fil via une application sur ordinateur ou téléphone portable. L'objectif est de permettre la récupération des données de l'environnement de la carte, provenant des capteurs ainsi que le pilotage des relais.

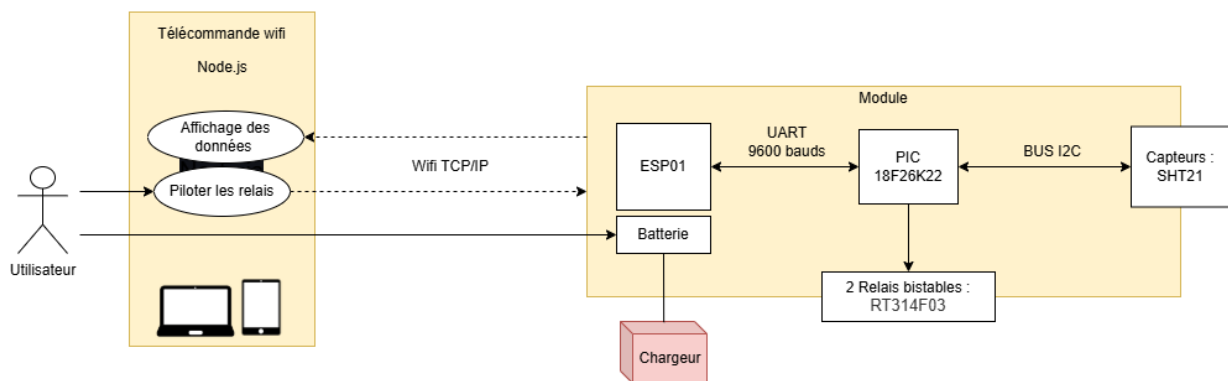


Figure 1 : Diagramme des cas d'utilisations

Ce projet s'inscrit dans le cadre du développement de compétences en électronique, électronique embarquée et programmation microcontrôleur. Elle présente aussi un intérêt vis-à-vis de l'internet des objets et des environnement connectés tels que les usines 5.0, qui sont équipées des robots, d'IA et d'une architecture réseau interne permettant la communication entre les machines et avec les opérateurs. Cette étude permet aussi de se confronter à des réalités futures, tels qu'un cahier des charges peu développé et la prise de décision dans les choix logiciels et techniques que l'on devra faire et justifier dans notre futur travail.

Ce projet sera effectué en majorité à l'IUT pendant des TP de 3h à 4h en moyenne et ce pendant toute l'année, pour pouvoir concilier les connaissances et compétences à acquérir avec le développement de notre projet en parallèle. Aussi pendant les périodes d'interruption pédagogiques nous pouvons avancer sur le projet tel que la programmation ou la réalisation d'améliorations.

1.2. Cahier des charges

Lors des premières séances de SAE, nous avons commencé par rédiger le cahier des charges afin de définir les spécifications techniques, les contraintes et les fonctionnalités attendues du projet.

Exigences Fonctionnelles :

- Acquisition des données de température et d'humidité.
- Commande de relais pour piloter des actionneurs.
- Communication sans fil via un module Wi-Fi ESP01 (ESP8266) imposé.
- Interface utilisateur permettant la visualisation des données et le contrôle des relais.

Exigences Techniques :

- Utilisation d'un microcontrôleur PIC18F26K22 imposé pour le contrôle du système.
- Intégration d'un module Wi-Fi ESP01 (ESP8266 imposé) pour la transmission des données.
- Alimentation autonome avec gestion optimisée de l'énergie.

Contraintes :

- Faible consommation énergétique.
- Compatibilité avec les protocoles domotiques standards.
- Robustesse et fiabilité du système pour un usage domestique.

Pour mieux formaliser et se représenter le cahier des charges, nous avons choisi de le présenter sous la forme d'un diagramme des exigences. La version ci-dessous a évolué par rapport à celle du début du projet, grâce aux différents retours des professeurs et aux améliorations que nous avons proposées.

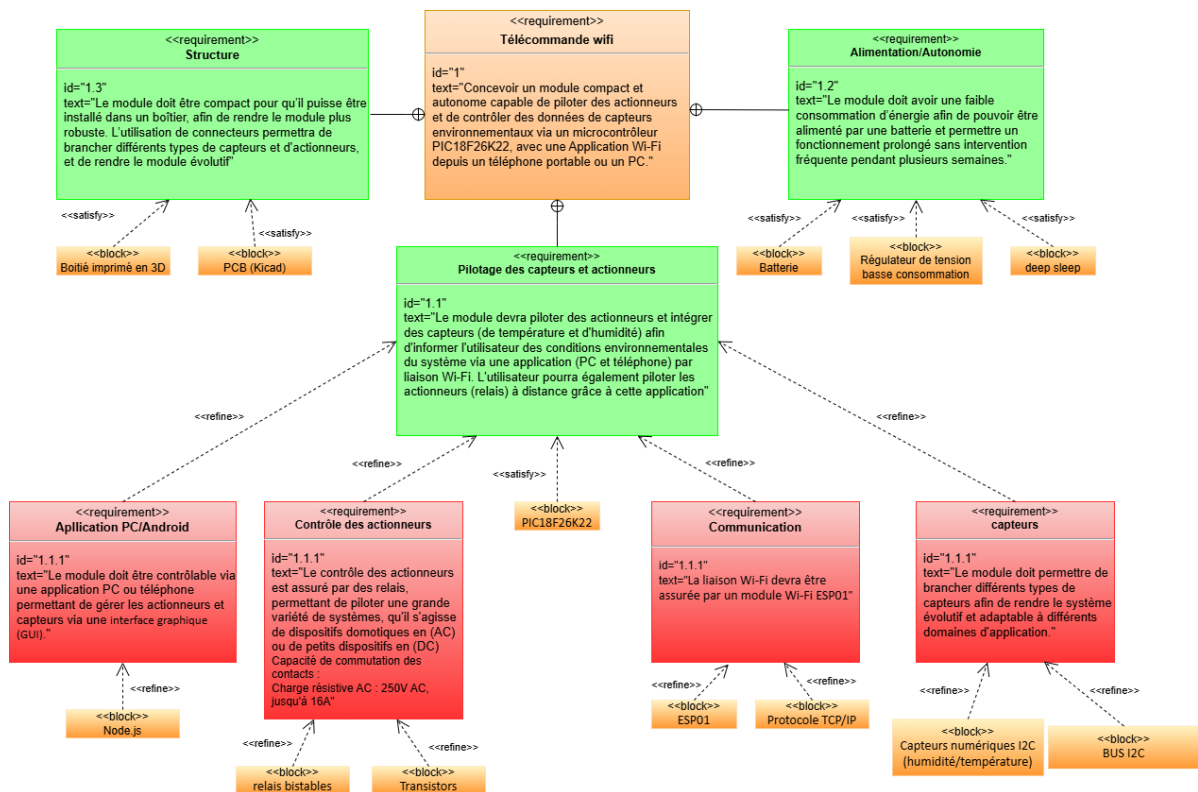


Figure 2 : Diagramme des exigences

1.3. Gestion de projet

Afin de garder une bonne organisation dans la réalisation de notre projet, nous avons dû faire un planning prévisionnel de type Gantt sur Excel pour répartir le temps attribué à chaque tâche et nous permettre de se répartir les tâches. Nous avons également complété ce document avec un planning réel pour vérifier l'évolution du projet.

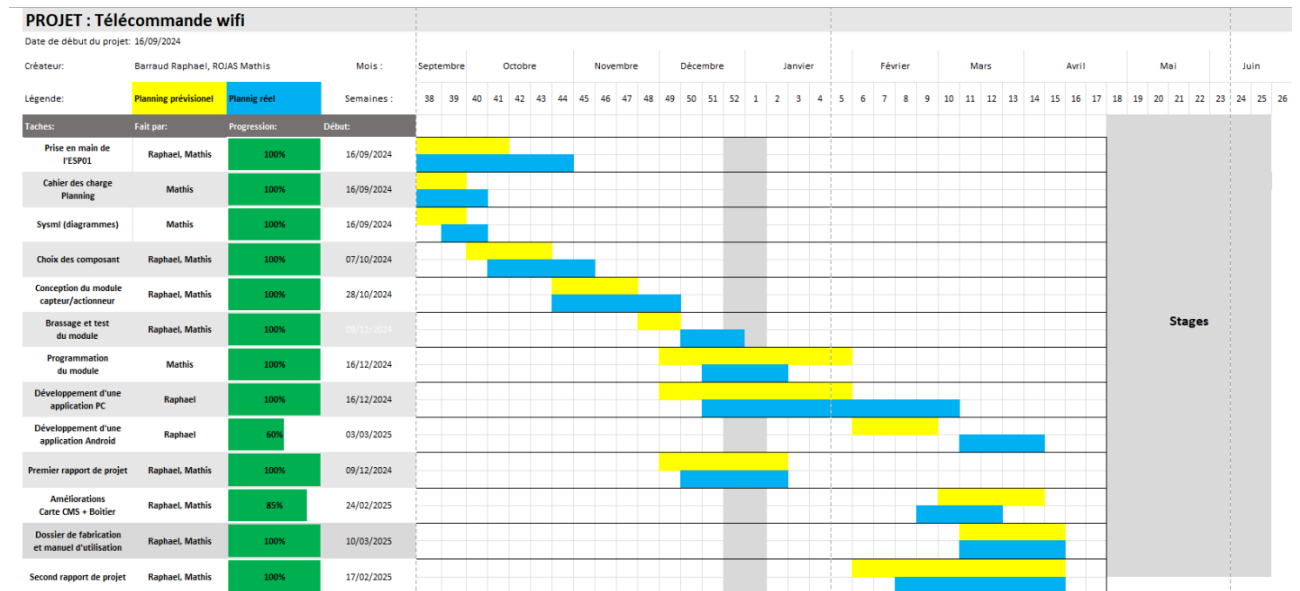


Figure 3 : Planning prévisionnel et réel (Gantt)

2. Etat de l'art

Lorsque l'on recherche sur internet esp-01, on peut retrouver des systèmes tel que le nôtre, qui possèdent 1 ou 2 relais, en 10A 230V par exemple avec un esp01, comme celui-ci-dessous, chez Ali express à 0,60€.

Cependant la plupart de ces produits ne sont pas équipés de capteurs et n'ont pas ou très peu de documentation technique sur les fonctionnalités implémentées tels que la programmation du pic ou encore sur les ajouts possibles. Il aurait sans doute été possible de partir de ce genre de base et de l'implémenter sur une carte électronique avec le PIC18F et un capteur.

3. Présentation de l'ESP01

L'ESP01 est un module wifi basé sur une puce ESP8266EX, conçu par Espressif Systems. Il est très populaire dans l'Internet of Things (Internet des Objets, IoT). L'ESP peut être programmé via divers environnements, il dispose d'un processeur 32 bits, d'une mémoire flash intégrée, et de plusieurs GPIO pour contrôler des périphériques. Idéal pour des applications comme l'automatisation domestique, la domotique ou les capteurs connectés, il correspond totalement aux besoins de notre projet.

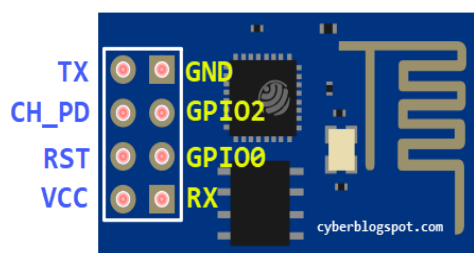


Figure 4 : Broches de l'ESP01

- La carte peut être alimentée en 3,3V.
- Les broches Tx et Rx sont utilisées pour la liaison série (UART).
- EN est la broche d'activation du module, active à l'état haut
- RST (Reset), active à l'état bas
- GPIO0 et GPIO2 à l'état haut pour fonctionner en mode UART
- GPIO0 à l'état bas et GPIO2 à l'état haut pour flasher le firmware.

L'ESP communique à l'aide de commande AT tels que « AT+Commande » par liaison série, voir la bibliographie pour des exemples de commande

4. Prise en main de la console et de l'ESP

4.1. La Chamberod-console pour linux

Nous avons commencé par utiliser la console de Monsieur Chamberod sous linux pour communiquer en UART avec notre ESP.

Voici la suite des actions à effectuer pour pouvoir l'utiliser :

- Copier la console dans un dossier personnel
- Charger les autorisations de communication à un port USB
 - `dtty`
- Ouvrir la console
- Ensuite il faut choisir le port USB ou notre module est connecté
 - F8 pour passer en USB
 - Shift + Fx pour se mettre sur le port x de communication
- Sélectionner la vitesse de communication avec les touches page-up ou page-down. Dans notre cas ça sera 115200 bauds car c'est la vitesse par défaut de l'ESP.
- Ensuite, nous pouvons modifier certains paramètres pour rendre l'affichage sur l'interface plus confortable et adapté aux besoins, tels que la désactivation de l'écho, l'ajout de retours à la ligne (CR+LF), etc.

4.2. Configuration de l'ESP et commande AT

La configuration par défaut de l'ESP01 pour la communication UART est la suivante :

- Baud rate : 115200
- Bits de données : 8
- Bit de parité : Aucune
- Bit de stop : 1
- Contrôle de flux : Aucun

Après avoir configuré la console avec les mêmes paramètres, nous pouvons tester la communication entre l'ordinateur et l'ESP01 via la liaison série en envoyant la commande **AT**. Le module Wi-Fi doit répondre par "**OK**", ce qui indique que la communication fonctionne correctement

Pour rendre notre module Wi-Fi plus économique en énergie, nous avons constaté lors du TP CONSO-PIC qu'une fréquence d'horloge plus basse permet de réduire considérablement la consommation du PIC.

La puissance consommée par un processeur dépend de la relation :

$$P = C \times V^2 \times f$$

V : tension d'alimentation,

f : fréquence d'horloge du processeur

C : capacité commutée effective.

Nous avons donc choisi de configurer la fréquence d'horloge du PIC à 1 MHz.

Concernant la liaison série du module UART, nous avons la possibilité de configurer la vitesse en modifiant la valeur du registre SPBRG. Selon la [formule du datasheet PIC18FK22 p272](#) $Baud\ Rate = \frac{FOSC}{64 \times (SPBRG + 1)}$, en définissant SPBRG à 0, nous obtenons un baud rate maximal de $\frac{1MHz}{64} = 15\,625\,bauds$. Nous avons donc opté pour une vitesse de 9600 bauds, car un baud rate plus faible permet de réduire les erreurs de communication, tout en ayant un impact négligeable sur la consommation d'énergie. En effet, bien qu'un baud rate plus faible puisse légèrement augmenter la consommation, cela reste très faible comparé aux gains d'énergie réalisés par la fréquence d'horloge réduite.

Nous avons donc configuré l'ESP de manière définitive avec la commande **AT+UART_DEF=9600,8,2,0,0** (9600 bauds, 8 bits de données, 2 bits de stop, pas de parité, pas de contrôle de flux).

(Pour comprendre l'intérêt des bits de stop, voir la partie programmation de la carte Wifi.)

5. Etablissement d'une communication

L'ESP8266 prend en charge plusieurs protocoles de communication réseau :

- **UDP/IP** : Communication rapide sans connexion, idéale pour les applications temps réel.
- **TCP/IP** : Communication fiable et ordonnée, idéale pour les applications nécessitant la sécurité des données.
- **SSL/TLS** : Sécurisation des communications avec chiffrement, utile pour des transactions sensibles.
- **HTTP/HTTPS** : Accès à des ressources web, idéal pour les requêtes API ou la récupération de données.
- **MQTT** : Messagerie légère pour les applications IoT.

Dans notre projets les deux protocoles qui nous correspondent le mieux sont UDP/IP et TCP/IP donc nous les avons testés pour choisir le protocole que nous allons choisir

Pour cela, il y a deux étapes à réaliser : mettre l'un des deux ESP en mode station (ESP2), celui que nous utiliserons sur la carte, et l'autre en mode point d'accès (ESP1) pour tester la communication. Voici la configuration que nous avons utilisée :

ESP1	ESP2
AT+CWMODE_DEF=2 (défini en point d'accès wifi)	AT+CWMODE=1 (défini en station)
AT+CWSAP_DEF="WIFIBAR","le_mot_de_pas",1,4,4,0	AT+CWJAP="WIFIBAR","mot_de_passe" ("SSID", "mot de passe") (connexion) → A faire après avoir créé le wifi
	AT+CWQAP (Se déconnecter d'un AP) AT+CWLAP (Liste des AP)

5.1. UDP/IP

ESP1	ESP2
AT+CIPSTART="UDP","192.168.4.1",1000,1002,0 (<type>, <remote IP>, <remote port>[, <UDP local port>, <UDP mode>) CONNECT	AT+CIPSTART="UDP","192.168.4.2",1002,1000,0 (<type>, <remote IP>, <remote port>[, <UDP local port>, <UDP mode>) CONNECT

OK	OK
AT+CIPSEND=5 OK > hello Recv 5 bytes SEND OK	+IDP, taille : message (réponse)

5.2. TCP/IP

ESP1	ESP2
AT+CIPMUX=1 (On autorise plusieurs connexions parallèles) AT+CIPSERVER=1,8080 (1 : Active le serveur,8080 : PORT)	AT+CIPSTART="TCP","192.168.4.1",8080 (Connexion au serveur)
AT+CIPSEND=ID,LENGTH OK > hello SEND OK	+IDP, taille : message (réponse)
+IDP, taille : message (réponse)	AT+CIPSEND=LENGTH OK > hello SEND OK
AT+CIPSERVER=0 (désactive le mode serveur) AT+CIPCLOSE=ID (ferme la connexion avec id, si id=0 ferme toutes les connexions)	AT+CIPCLOSE (fermer la connexion)

Après avoir testé les deux protocoles, nous avons décidé d'utiliser le protocole **TCP/IP**, car il s'avère plus simple à mettre en place et à configurer. En mode serveur, notre ESP peut gérer plusieurs connexions sans avoir à connaître l'adresse IP de chaque station connectée. Ce mode nécessite seulement un seul port de communication, ce qui simplifie la gestion des connexions.

De plus, la commande **AT+CIPSEND=5,<longueur_données>** permet d'envoyer des données simultanément à tous les clients connectés au serveur, ce qui rend la gestion des interactions plus fluide et efficace, en particulier lorsqu'il y a plusieurs utilisateurs contrôlant la station en même temps.

Le protocole **TCP** présente également des avantages en termes de sécurité et de contrôle des erreurs. Il garantit une transmission fiable des données grâce à des mécanismes de retransmission et de vérification d'intégrité, ce qui améliore la stabilité du système. Toutefois, il consomme légèrement plus de ressources et peut entraîner une charge plus importante sur le réseau, mais ces inconvénients sont compensés par sa fiabilité et sa robustesse dans des environnements nécessitant une communication continue.

6. Choix des composants principaux

6.1. Capteurs de température

Après discussion avec Monsieur Chamberod, il nous a orientés vers des capteurs numériques, afin de simplifier sa mise en place, sans nécessiter de circuit électronique pour le conditionnement du signal,

ni l'utilisation du convertisseur analogique-numérique (CAN) du PIC. De plus, cette solution est moins coûteuse et permettra de rendre le système évolutif, en pouvant utiliser différents capteurs numériques similaires. Ainsi, nous avons cherché un capteur numérique capable de mesurer à la fois la température et l'humidité :

Capteurs	SHT21	DTH11	HIH6030-021-001	BME280
Plage de température	-40°C à 125°C $\pm 0,3^\circ\text{C}$	0°C à 50°C $\pm 2^\circ\text{C}$	-40°C à 85°C $\pm 0,5^\circ\text{C}$	-40°C à 85°C $\pm 1^\circ\text{C}$
Plage d'humidité	0% à 100% $\pm 2\%$	20% à 90% $\pm 5\%$	0 à 100% $\pm 4,5\%$	0% à 100% $\pm 3\%$
Plage de Pression				300 à 1100 hPa ± 1
Tension d'alimentation	2,1V à 3,6V	3V et 5V	2,3V à 5,5V	1,8V à 3,6V
Interface	I2C	OneWire	I2C	I2C et SPI
Prix	Environ 5-6€	Environ 4-6€	Environ 11,27 €	Environ 10-15 €

Nous avons choisi le SHT21, qui correspond le mieux à nos critères : il peut fonctionner en 3,3V, offre une bonne précision, utilise l'interface I2C et couvre une large plage de température. Cependant, une carte électronique doit être réalisée, car il s'agit d'un composant CMS, afin d'éviter que le capteur mesure la température de la carte elle-même. Par ailleurs le capteur est dans un boîtier DNF (sans pattes) ce qui va rendre le brasage du composant un plus complexe. Ce capteur est disponible chez [RS](#) à 4,965 € HT.

6.2. Relais

Puisque notre ESP fonctionne en 3,3V et que le PIC18F26K22 ainsi que le capteur SHT21 peuvent également fonctionner sous cette tension, nous avons décidé de concevoir notre carte entièrement en 3,3V. Cela permet de réduire la consommation énergétique et les pertes. Tous les autres composants devront donc être compatibles avec cette tension.

Pour les relais, il nous était imposé qu'ils aient un pouvoir de coupure d'au moins 10A sous 230VAC afin de permettre le branchement d'appareils du quotidien. Pour leur sélection, nous avons retenu les critères suivants :

- **Tension de bobine** : 3VDC, pour être compatible avec le reste de la carte.
- **Bistable** : pour que le relais ne consomme de l'énergie que pendant les phases de commutation, ce qui réduit considérablement la consommation énergétique globale.
- **Deux bobines** : pour faciliter l'inversion de l'état du relais.

Cependant, après de nombreuses recherches, sur les sites « où l'IUT fait ses achats » il s'est avéré difficile de trouver des relais bistables avec un courant de contact d'au moins 10A. Finalement il s'est avéré que en allant sur d'autres sites (Ali express), on pouvait trouver exactement ce que l'on cherchait, un relai bistable, à 2 bobines, 3,3VDC, avec un pouvoir de coupure de 16A à 230V. Ce sont les RT314F03 de SCHRACK à [4€49 chez Ali express](#).

6.3. Transistors

Pour le choix des transistors, le premier critère était qu'ils puissent fonctionner en mode bloqué/saturé, qu'ils soient capables de laisser passer un courant de 200 mA et qu'ils aient une tension de saturation V_{CEsat} suffisamment faible.

Nous avons fixé une chute de tension maximale tolérable d'environ 1 V, soit une $V_{CEsat} = \frac{1}{2} = 0,5V$ pour chaque transistor. Ensuite, nous avons dû choisir la technologie appropriée à utiliser pour les transistors :

Technologie	Commande	$R_{DS(on)}$ ou V_{CEsat}	Gain h_{FE}	Avantages	Inconvénients
MOSFET	Commande en tension V_{GS}	Moyen $R_{DS(on)}$: 3 à 5 Ω	Non applicable (commande tension)	Très faible dissipation en conduction, rapide	Nécessite un MOSFET logique compatible 3,3 V
Bipolaire	Commande en courant I_B	Faible V_{CEsat} : $\approx 0,2$ à 0,3 V	Gain h_{FE} : ≈ 10 à 200	Simple à piloter, faible coût	Courant de base requis, dissipation plus élevée

Pour les MOSFET, la résistance de drain-source à l'état passant $R_{DS(on)}$ étant relativement élevée pour des tensions de commande de 3,3 V et un courant I_D de seulement 200 mA, nous avons décidé d'utiliser des transistors bipolaires.

Nous avons donc sélectionné, pour les transistors NPN, des **2SCR514RTL** à 0,645 € HT pour un lot de 5 chez Farnell. Ces transistors permettent d'obtenir un I_C supérieur à 200 mA avec une tension de saturation V_{CEsat} relativement faible, de l'ordre de 0,3 V. De plus, ils offrent un gain h_{FE} stable, supérieur à 200, pour un courant de 200 mA à température ambiante.

$$\text{NPN: } R_B = \frac{V_{CC} - V_{BE}}{I_B} = \frac{V_B - V_{BE}}{\frac{I_C}{h_{FE}}} = \frac{3,3 - 0,8}{\frac{200 \times 10^{-3}}{200}} \approx 2500 \Omega$$

On prendra donc des résistances de 2,2 k Ω pour les R_B des transistors.

6.4. Régulateur de tension

La consommation de courant maximale que nous pouvons avoir lorsque tous les composants fonctionnent simultanément est estimée de la manière suivante :

Relais (bobines) : $2 \times 200 \text{ mA} = 400 \text{ mA}$

Transistors (base) : 2,25 mA

Capteur de température et d'humidité en mesure [p.2](#): 0,3 mA

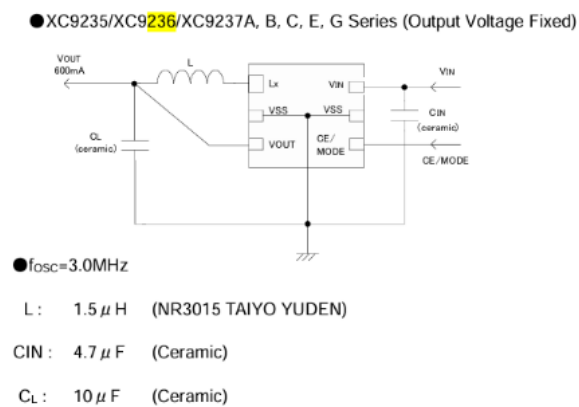
Module ESP01 (émission + réception) [p.17](#) : 190 mA

Microcontrôleur PIC18F26K22 (1 MHz, $V_{dd} = 3,3V$) [p.421](#): 0,5 mA

Calcul de la consommation totale :

$$400 + 2,25 + 0,3 + 190 + 0,5 \approx \mathbf{593 \text{ mA}}$$

Pour alimenter l'ensemble de notre système en 3,3V, nous devons utiliser un régulateur de tension à découpage basse consommation capable de fournir au maximum 600 mA. Ce régulateur permettra de stabiliser la tension et protéger notre circuit contre les surtensions. Nous avons donc choisi le **XC9236A33DMRG** disponible chez [farnell à 1,740 € HT](#)



- **Tension de sortie** : 3,3 V
- **Tension d'entrée** : comprise entre 4 V et 6 V
- **Courant de sortie maximal** : 600 mA
- **Courant d'alimentation** : 34 μA
- **Bobine utilisée** : NR3015

Figure 5 : Schéma électrique du câblage du régulateur de tension

7. Développement des 2 cartes

Nous avons décidé de séparer la carte Wi-Fi de la carte capteur afin de pouvoir facilement remplacer le capteur en cas de panne ou utiliser d'autres types de capteurs à l'avenir. Cette séparation permet également d'éviter que le capteur mesure la température de la carte Wi-Fi, ce qui pourrait fausser les relevés de température. De plus, cette configuration offre la possibilité d'ajouter plusieurs capteurs et de créer un bus I²C.

7.1. Carte capteur

Simple carte avec capteur en CMS et broches en traversant pour la connecter à la carte mère.

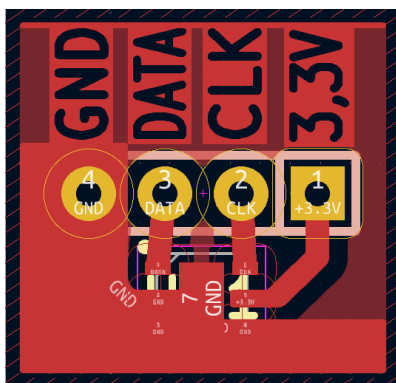


Figure 6 : PCB version 1

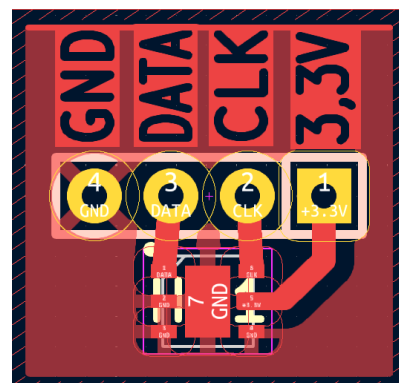


Figure 7 : PCB version 2 (corrigé)

Sur la Figure 6, la face supérieure est en rouge. On peut observer que l'on a lié le plan de masse à l'aide de carrés rouges, on apprendra lors de la mise en place du composant au CEDMS qu'il ne faut pas le faire car la grande zone en cuivre risque d'attirer le composant quand on le soudera.

Sur la face supérieure on a le capteur SHT21, les pistes et le plan de masse qui fait aussi office de sonde de température de par sa taille.

7.2. 2^{ème} Carte wifi

Dans la deuxième partie de ce projet, nous avons décidé de refaire la carte WiFi en utilisant un maximum de composants CMS afin de réduire considérablement sa taille et de minimiser son impact

sur l'environnement. Nous y avons également apporté plusieurs améliorations, notamment l'ajout d'un bus I2C pour rendre la carte évolutive et permettre l'intégration de plusieurs capteurs. De plus, les relais ont été remplacés par des relais bistables à deux bobines, ce qui a permis de supprimer les ponts H ainsi que leurs composants associés (transistors, résistances et diodes). Toutes les informations sur la première version de la carte WiFi sont disponibles en annexe.

Cette carte est constituée de plusieurs parties. La première partie est dédiée à la puissance, avec deux relais bistable piloté par des transistors bipolaires NPN. La seconde partie intègre le microcontrôleur (PIC18F26K22), accompagné d'un socket compatible avec le programmeur PICKIT 3, ce qui facilite le développement et le débogage du PIC. Enfin, la carte comprend une section capteur/communication, avec un capteur numérique SHT21 pour la mesure de la température et de l'humidité, ainsi qu'un connecteur pour brancher l'ESP qui va assurer la transmission des données via Wi-Fi.

7.2.1. Schéma électrique

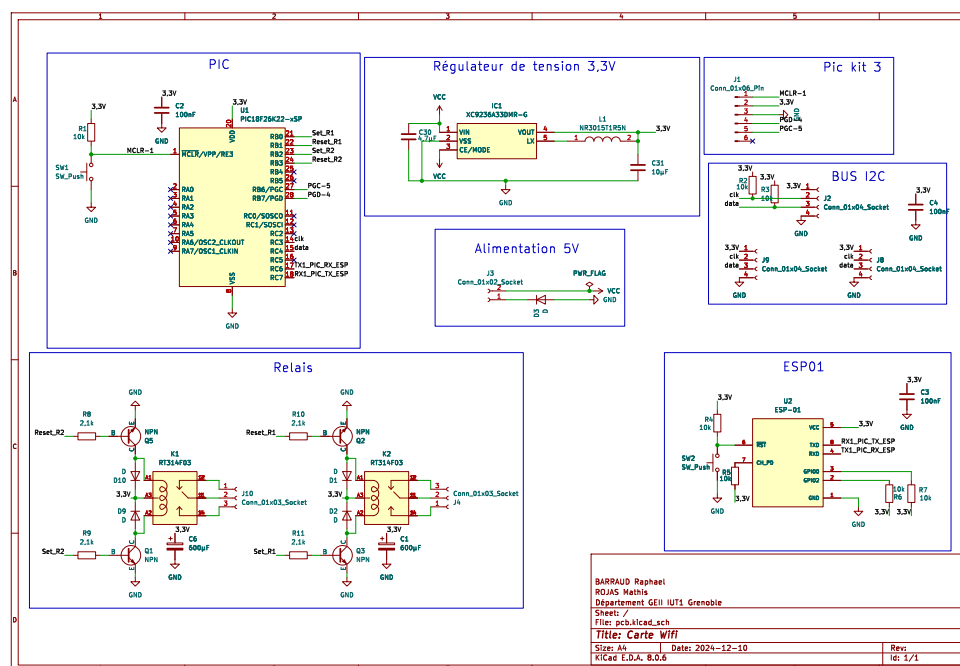
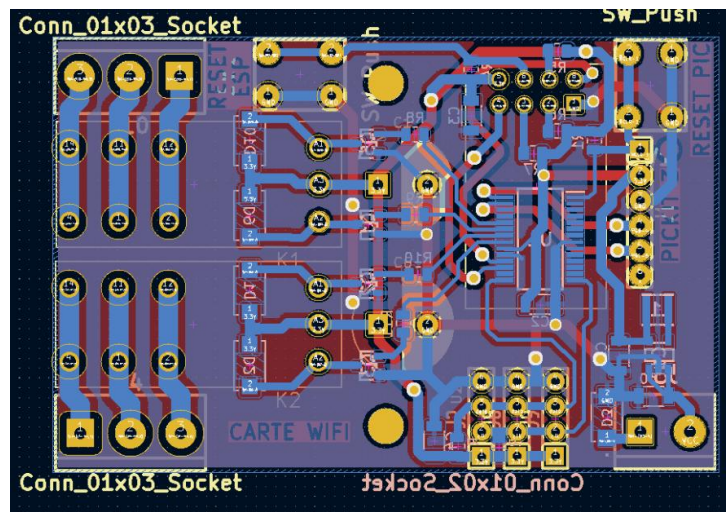


Figure 8 : Schéma électrique

7.2.2. Routage

Pour le routage, nous avons essayé de respecter au maximum les bonnes pratiques vues en cours sur le **routage et la CEM** :

- Les tailles de pistes et l'isolation en fonction des courants
- Nous avons placé la piste VCC sur la face opposée au GND, directement au-dessus du plan de masse, afin de réduire l'impédance parasite et d'améliorer la qualité de l'alimentation.
- La séparation entre la partie puissance (relais) et la partie numérique a été respectée pour limiter les perturbations électromagnétiques sur les signaux sensibles.
- Des capacités de découplage ont été placées au plus près des composants.



Rouge : face avant
Bleu : face arrière

Figure 9 : Routage de la carte

7.2.3. Réalisation de la Carte Wifi

La finesse des pistes du PIC CMS (0,4 mm) rendait leur réalisation compliquée avec une gravure chimique au perchlorure de fer. De plus, les délais de tirage étant trop longs. Et disposant d'une CNC Badog Artisan (CNC suisse avec une précision de 10 μm), nous avons décidé de fabriquer nous-mêmes la carte en usinant le cuivre.

À partir des fichiers Gerber, nous avons utilisé **FlatCAM** pour générer les trajectoires d'outil nécessaires à la gravure et au perçage et de convertir les données du circuit en G-code, exploitable par la CNC.

Paramètres optimisés pour FlatCAM et la CNC :

- Outil : Mèche javelot 0,1 mm, 10°
- Profondeur de coupe (Cut depth) : 0,06 mm
- Nombre de passes : 3 avec un overlap de 20 %
- Vitesse d'avance (Feed rate X/Y) : 150 mm/min
- Vitesse d'avance (Feed rate Z) : 100 mm/min
- Vitesse de la broche (Spindle Speed) : 10 000 tr/min (167 Hz, car une paire de pôles)

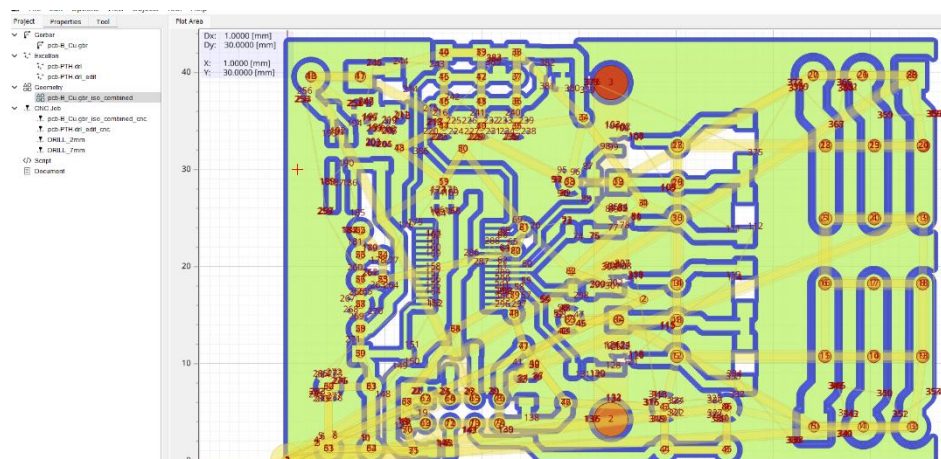


Figure 10 : FlatCAM de la carte

Avant de lancer l'usinage, un étalonnage de Z a été effectué à l'aide de **AutoLeveller**. Cette étape est cruciale pour corriger les irrégularités de la plaque de cuivre et garantir une profondeur de coupe uniforme, évitant ainsi des pistes trop fines ou mal gravées voir Figure 15 et Figure 16. Une fois le nivellement terminé, nous avons utilisé **Mach3** pour piloter la CNC et réaliser le PCB voir Figure 17, Figure 18 et Figure 19.

8. Programmation de la Carte WIFI

8.1. Présentation globale du programme

Le programme de la carte Wifi a été développé en C sur MPLAB X. Son objectif principal est d'assurer une transmission efficace afin que la commutation des relais se fasse rapidement et de permettre l'envoi périodique des mesures du capteur à l'utilisateur, tout en minimisant la consommation d'énergie.

Pour cela, la communication entre le microcontrôleur et l'ESP se fait via une liaison UART, tandis que les mesures des capteurs sont réalisées via un bus I2C et envoyées à l'ESP toutes les cinq secondes grâce à l'utilisation du Timer0, uniquement lorsqu'un client est connecté.

La structure du programme repose sur la gestion des interruptions, permettant de réagir en « temps réel » aux commandes de l'application pour modifier l'état des relais. De plus, lorsque aucun client n'est connecté au module Wifi, le microcontrôleur est mis en veille afin de réduire la consommation électrique.

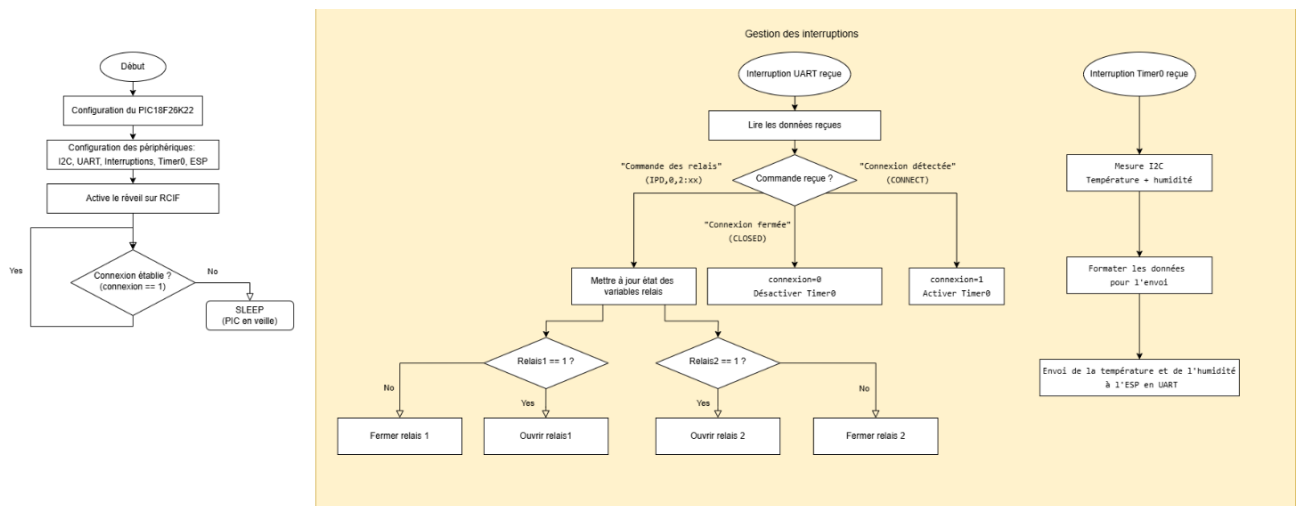


Figure 11 : Algorithme représentant la structure du code

8.2. Gestion des interruptions

Deux interruptions principales sont mises en place :

- Une interruption UART en haute priorité pour éviter toute perte d'information envoyée par l'ESP (utilisateur).
- Une interruption Timer0 en basse priorité pour l'acquisition des mesures et leur envoi au client.

L'interruption UART permet au microcontrôleur de réagir immédiatement aux commandes reçues via le serveur TCP. Lors de la réception d'une trame, le programme vérifie de quelle commande il s'agit :

une connexion, une déconnexion ou une demande de commutation des relais. Dans ce dernier cas, la variable d'état des relais est mise à jour, et ces derniers sont commutés en fonction de cette nouvelle valeur.

Pour optimiser la consommation d'énergie, le microcontrôleur se réveille uniquement lors de la réception d'une commande UART. Il reste actif après une connexion (commande **CONNECT**) et retourne en veille à la déconnexion (**CLOSED**).

Cependant, le réveil du microcontrôleur n'étant pas instantané, les premiers bits reçus sont parfois corrompus. Une méthode de resynchronisation a été mise en place :

1. Désactivation temporaire de l'UART dès la détection d'une réception.
2. Application d'un délai mesuré à l'oscilloscope pour stabiliser le système (voir annexes 12.2).
3. Vidage du registre de réception avant réactivation de l'UART.
4. Relecture de la trame à partir du deuxième octet.

Pour améliorer la stabilité, l'ESP est configuré avec deux bits de stop, ce qui réduit les risques de corruption de trame malgré la perte du premier caractère, sans altérer les informations essentielles.

L'interruption Timer0 déclenche toutes les 5 secondes l'acquisition des mesures de température et d'humidité via I2C (capteur SHT21). Les valeurs sont ensuite envoyées au client via l'ESP.

9. Télécommande wifi (Application PC)

On utilise Node.js pour l'interface, avec un serveur en JavaScript et des pages en HTML/CSS. L'appli repose sur trois pages web : une pour le choix du réseau, une pour la connexion et une dernière pour l'interaction entre le boîtier et l'utilisateur.

J'ai eu plusieurs difficultés. D'abord, choisir un logo et des couleurs pour la page web. J'ai utilisé l'IA pour générer un logo et en déduire la palette de couleurs. Ensuite, en scannant les réseaux, tous ne s'affichaient pas. La solution : utiliser l'onglet Windows de recherche des réseaux, puis relancer le scan sur la page web. J'ai tenté d'intégrer le protocole de scan Windows, mais cela n'a rien changé et a alourdi le code. Ce problème n'existe peut-être pas sous Linux.

Mon application semblait fonctionnelle sur mon PC, mais ne marchait pas sur d'autres appareils Windows. J'ai passé des semaines à modifier le code, parfois en le rendant inutilisable. J'ai finalement compris qu'elle ne fonctionnait que lorsque mon serveur Node.js était actif en arrière-plan. Je cherche encore un moyen de faire démarrer un serveur Node.js depuis l'application, même sans Node.js installé sur l'ordinateur.

9.1.1. Première page



La première page est la page qui va gérer le scan des réseaux, la récupération du SSID et le passage à la page d'après. On peut aussi entrer un SSID manuellement. Une fois que la page a récupéré un SSID (qu'il soit entré manuellement ou sélectionné dans les réseaux scannés), elle affiche le bouton pour passer à la page d'après.

Figure 12 : Première page avec un scan effectué

9.1.2. Seconde page

Figure 13 : Page de connexion avec le mot de passe déjà entré

La deuxième page gère l'entrée du mot de passe et la tentative de connexion. Un layer assombrit l'arrière-plan pour empêcher d'autres actions avant la fin du processus. En cas d'échec, on revient à la page de connexion avec l'option de retourner au choix du SSID. Sinon, on passe à l'étape suivante. L'application affiche aussi une alerte pour les tentatives sans mot de passe.

9.1.3. Troisième page

Figure 14 : Troisième page après connexion au TCP et réception d'informations

La troisième page comprend un bouton pour se connecter au TCP. Une fois connecté, ce bouton disparaît pour laisser place à celui de déconnexion ainsi qu'aux boutons d'ouverture et de fermeture des deux relais. Lors de la déconnexion, on revient à l'affichage initial et on quitte le serveur TCP de la carte. Si la page est fermée alors que la connexion TCP est active, l'application se déconnecte automatiquement pour éviter de saturer le serveur et garantir que l'ESP communique uniquement avec le premier utilisateur connecté

10. Consommation électrique

Estimation théorique

- Consommation du PIC = $0,5mA$
- Consommation moyenne ESP en veille = $0,01mA \times \frac{4,5 \text{ secondes}}{5 \text{ secondes}} s = 9\mu A$
- Consommation moyenne réception plus transmission Wi-Fi = $200mA \times \frac{0,5 s}{5 s} = 20mA$

Ainsi, la consommation moyenne sur 1 seconde est de $0,5 + 0,009 + 20 \approx 20,51 mA$ (1)

Consommation lors de la commutation d'un relais : $201 mA$ pendant $10 ms$

Consommation en mode repos (ESP01 p.17 + PIC18F26K22 p.418 + régulateur de tension p.7) :

$$10\mu A + 0,06\mu A + 35\mu A = 45,06\mu A \text{ (3)}$$

Hypothèses :

- Le système fonctionne 30 minutes par jour avec les relais qui commutent 20 fois par jour.
- Le reste du temps, le système est en mode repos.
- Autonomie souhaitée : 2 mois.

Nous pouvons estimer la charge totale (Q) de la batterie nécessaire pour un mois de fonctionnement :

$$Q = 2 \times 31 \times \left(20,51 \times 10^{-3} \times \frac{3600}{2} + 20 \times 201 \times 10^{-3} \times 10 \times 10^{-3} + 0,045 \times 10^{-3} \times 23,5 \times 3600 \right) \\ = 2527 C \text{ soit } 0,7 Ah$$

Observations pratiques

- Sur la première carte Wi-Fi, la consommation moyenne était de **120 mA**.

- Après optimisation matérielle et logicielle, la consommation moyenne a été réduite à **90 mA**.
- Cependant, en mode serveur TCP, l'ESP ne peut pas être mis en light sleep ou modem sleep, et le deep sleep entraîne la perte du comportement souhaité.
- À **90 mA**, une batterie de **15 Ah** serait requise pour 1 semaine d'autonomie, ce qui est trop élevé pour un système embarqué. Il est nécessaire d'explorer d'autres technologies ou modèles pour réduire la consommation énergétique et assurer une meilleure autonomie.

11. Conclusion

Travail réalisé :

Nous avons conçu une carte électronique optimisée en technologie CMS, permettant de réduire l'encombrement, de minimiser la consommation énergétique et de réduire son impact sur l'environnement. Le programme de la carte Wifi a été amélioré pour optimiser la gestion des relais et la transmission des données, tout en intégrant un mode veille intelligent afin de limiter la consommation lorsque le système est inactif et en réduisant la puissance de transmission de L'ESP.

En parallèle, une application PC a été développée. Cette interface permet d'interagir avec la carte Wifi, de surveiller les mesures des capteurs. Cependant cette interface n'est pas pleinement fonctionnelle sur tous les support souhaités (Windows, Linux, Android...).

Travail restant à effectuer :

Pour aller plus loin, il serait pertinent d'explorer de nouvelles technologies de communication Wifi afin de réduire encore la consommation énergétique. L'application, bien que fonctionnelle, doit être finalisée, enrichie de nouvelles fonctionnalités et de compatibilité avec d'autres supports. D'un point de vue matériel, l'ajout d'une batterie permettrait de rendre le système autonome, et un circuit de charge intégré faciliterait son alimentation via une source externe. Enfin, un boîtier sera conçu sur SolidWorks pour protéger l'électronique et imprimé en 3D.

12. Bibliographie

12.1. Datasheets des composants

[Datasheet ESP826](#)

[Datasheet ESP01](#)

[Datasheet PIC 18F26K22](#)

[Datasheet SHT21\(capteur d'humidité et de température I2C\)](#)

[Datasheet Relais bistable 3V DC à 1 bobine\(1^{ère} carte\)](#)

[Datasheet Relais bistable 3V DC à 1 bobine\(2^{ème} carte\)](#)

[Transistor bipolaire NPN](#)

[Transistor bipolaire PNP](#)

12.2. Ressources utilisées pour le développement et la programmation du projet

[Basic AT Commands](#)

[Wi-Fi AT Commands](#)

[TCP/IP AT Commands](#)

[ESP8266 AT Instruction Set](#)

[EXEMPLES UDP](#)

[Boot Mode Selection ESP8266](#)

[Documentation Node.js](#)
[Exemple de programme I2C](#)

13. Annexes

Figure 1 : Diagramme des cas d'utilisations	3
Figure 2 : Diagramme des exigences	4
Figure 3 : Planning prévisionnel et réel (Gantt)	5
Figure 4 : Broches de l'ESPO1	5
Figure 5 : Schéma électrique du câblage du régulateur de tension	11
Figure 6 : PCB version 1	11
Figure 7 : PCB version 2 (corrigé)	11
Figure 8 : Schéma électrique	12
Figure 9 : Routage de la carte	13
Figure 10 : FlatCAM de la carte	13
Figure 11 : Algorithme représentant la structure du code	14
Figure 15 : Première page avec un scan effectué	15
Figure 16 : Page de connexion avec le mot de passe déjà entré	16
Figure 17 : Troisième page après connexion au TCP et réception d'informations	16
Figure 15 : nivelage du plateau (AutoLeveller)	18
Figure 16 : système physique pour l'AutoLeveller	18
Figure 17 : usinage du PCB avec la CNC	19
Figure 18 : Résultat	19
Figure 19 : Mach3	19
Figure 20 : Vue de dessus de la carte	19
Figure 21 : Vue de dessous avec le PIC CMS	19

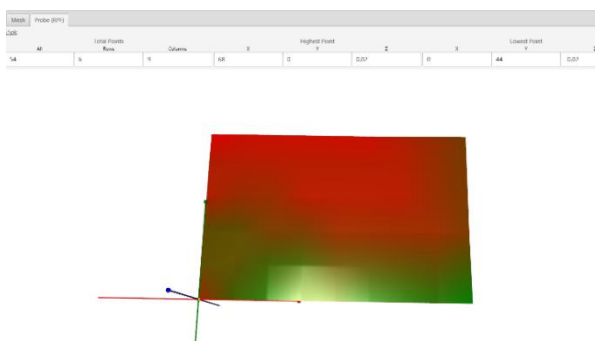


Figure 15 : nivelage du plateau (AutoLeveller)

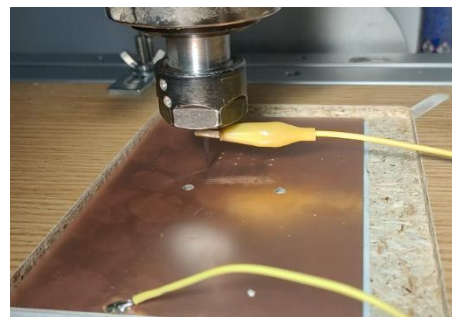


Figure 16 : système physique pour l'AutoLeveller

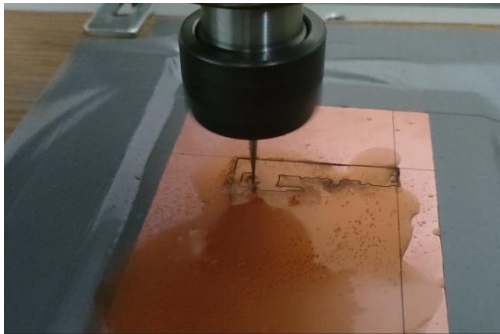


Figure 17 : usinage du PCB avec la CNC

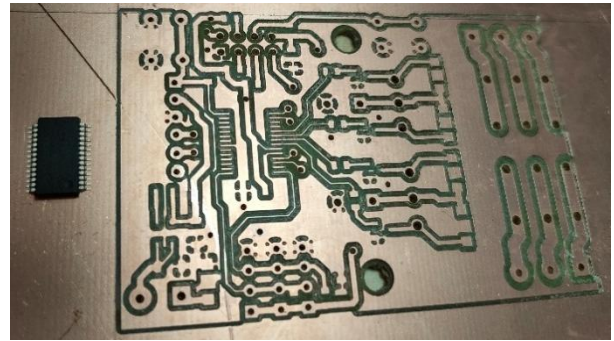


Figure 18 : Résultat

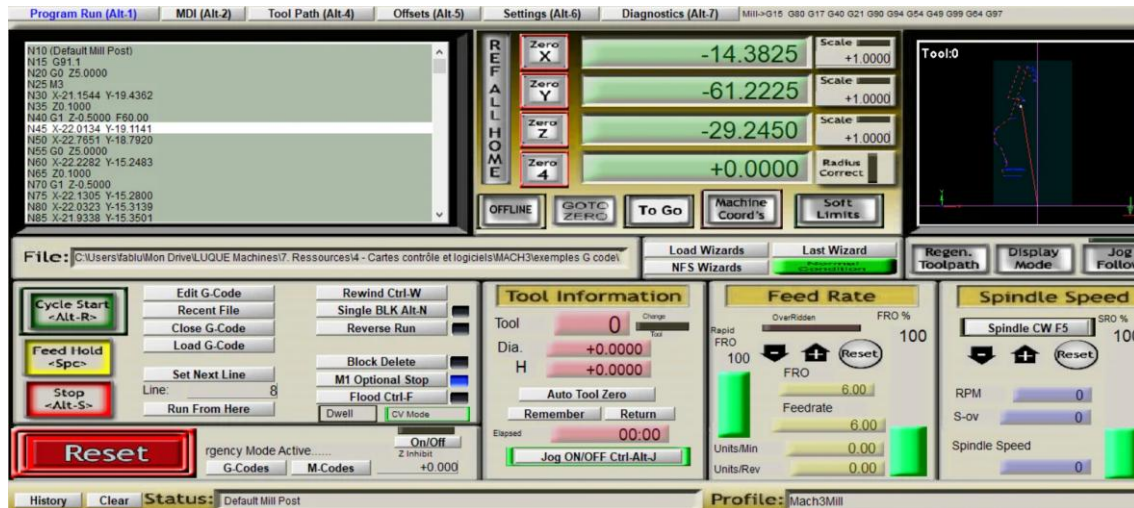


Figure 19 : Mach3

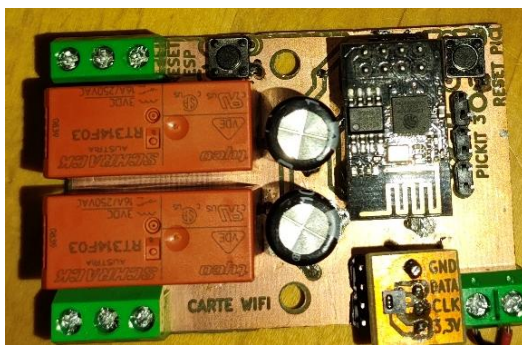


Figure 20 : Vue de dessus de la carte

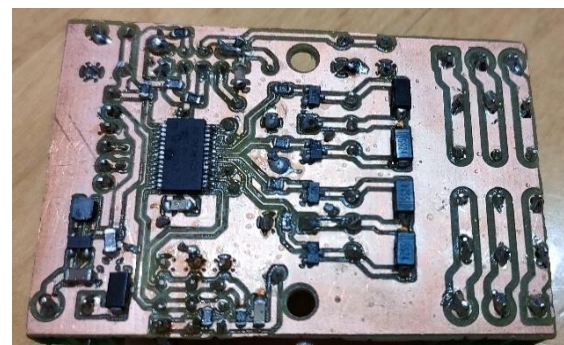
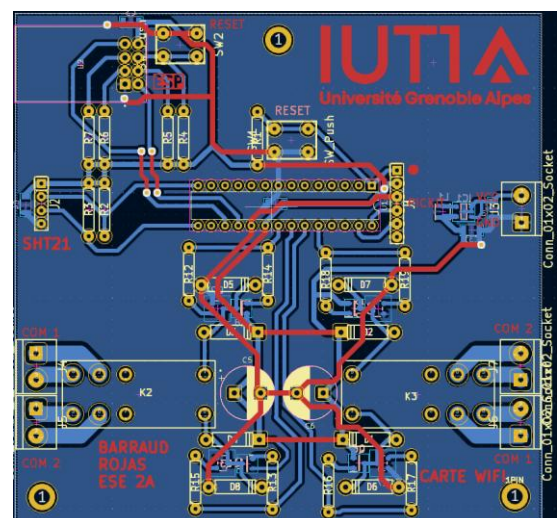
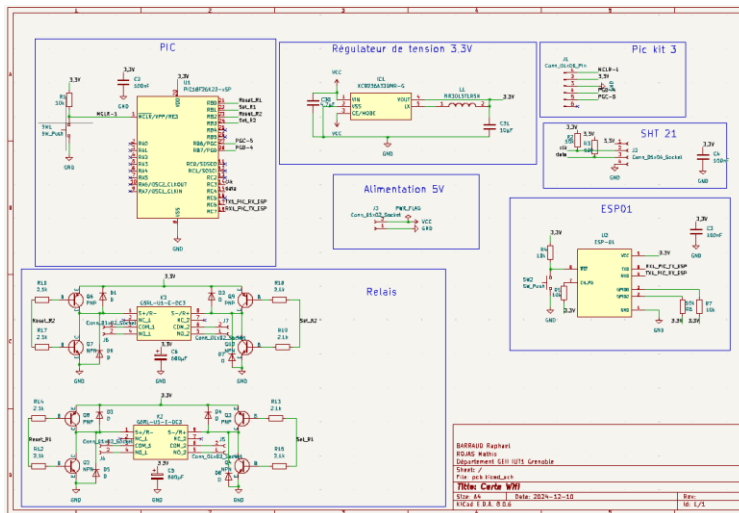


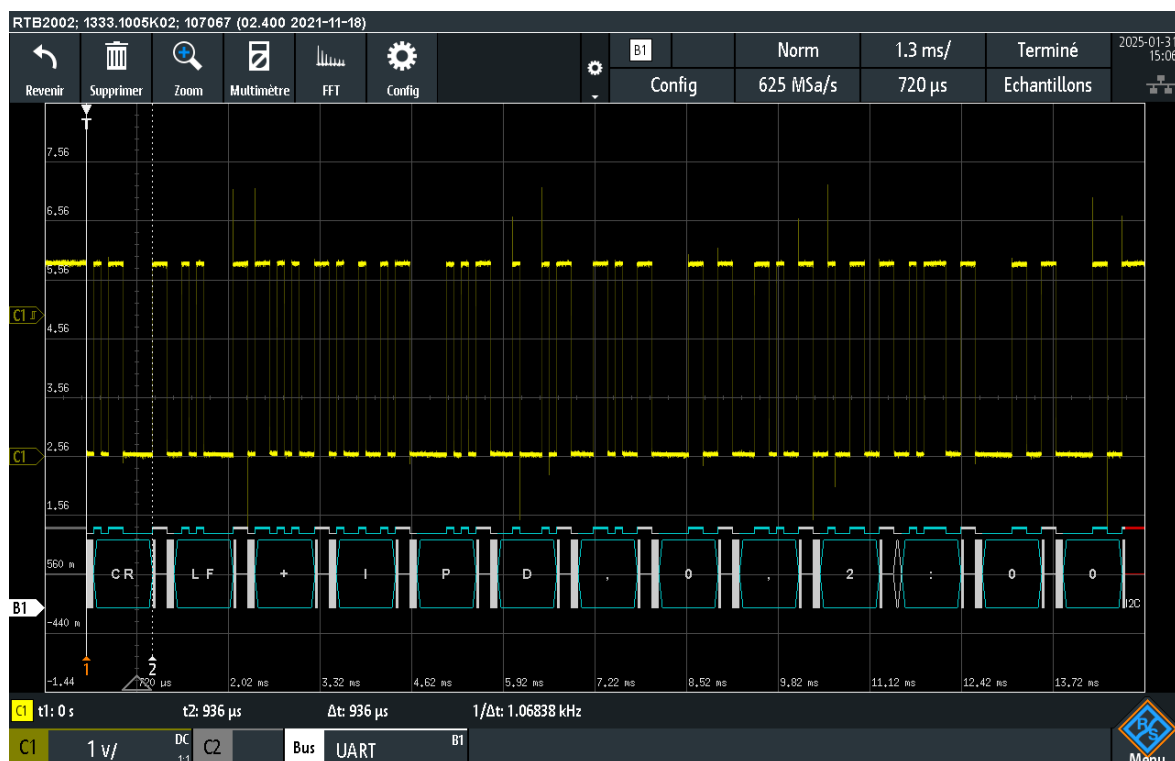
Figure 21 : Vue de dessous avec le PIC CMS

13.1. Schéma électrique et PCB de la 1^{ère} Carte Wifi





13.2. Communication UART entre l'ESP et le PIC18F26K22



13.3. Programme C de la carte WIFI (PIC18F26K22)

```
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <xc.h> //Fichier
//Fichier générique pour tous les Pics
#include "Uart.h"
```

```
// Remarque, liste des pragma config dans
C:\Program
Files\Microchip\xc8\v1.35\docs\chips
#pragma config WDTCN=OFF //Watchdog
contrôlé par SWDTEN
#pragma config WDTPS=512
//Postdiviseur Watchdog -> période 2s
environ
#pragma config PWRTE=OFF //Power up
timer désactivé
#pragma config BOREN=OFF //
```

```

#pragma config BORV=250          //Seuil BOR
2.5V
#pragma config MCLRE=EXTMCLR      //Master
reset externe
#pragma config DEBUG=OFF         //Mode debug
désactivé
#pragma config LVP=OFF           //Mode
programmation Low Voltage désactivé

#pragma config IESO=OFF
#pragma config PRICKEN=OFF       //primary
clock
#pragma config FCEN=OFF         //fail-safe
clock monitor
#pragma config FOSC=INTIO67
//Oscillateur interne

//Fréquence d'horloge choisie: (en MHz)
#define FCLK 1
#define XTAL_FREQ 1000000        //define
nécessaire pour l'utilisation des fonctions
__delay

#if
(FCLK==1)|| (FCLK==2)|| (FCLK==4)|| (FCLK==8)||
(FCLK==16)
#pragma config PLLCFG=OFF        //PLL
désactivée
#endif

#if (FCLK==32)|| (FCLK==64)
#pragma config PLLCFG=ON         //PLL
désactivée
#endif

//Définition des entrées/sorties du système
#define Reset_R1 LATBbits.LB1    //commande
ouvrir relai 1
#define Set_R1 LATBbits.LB0      //commande
fermée relai 1
#define Reset_R2 LATBbits.LB3    //commande
ouvrir relai 2
#define Set_R2 LATBbits.LB2      //commande
fermée relai 2

const char version[] @ 0xF00000 = "V1.00 -
LedK22 - 01/10/2023";

//Registre d'état global
volatile unsigned char STATM @ 0x060;
typedef union {
    struct {
        unsigned TEF : 1;          //B0: à
1, indique un top d'échantillonnage
        unsigned VaON : 1;         //B1: à
1, relai alim activé
        unsigned : 6;              //
    };
} STATMbits_t;
STATMbits_t STATMbits @ 0x060;
//Déclaration de STATM bit à bit

#define TEF_bit STATMbits.TEF
#define VaON_bit STATMbits.VaON

```

```

//Variables de fonctionnement du module en
Banque 0:
unsigned short cpt @0x061;

//-----
-----

// Fonction pour ouvrir le relai 1
void R1_ouvert(void)
{
    Set_R1=1;
    __delay_ms(10);
    Set_R1=0;
}

// Fonction pour fermer le relai 1
void R1_fermée(void)
{
    Reset_R1=1;
    __delay_ms(10);
    Reset_R1=0;
}

// Fonction pour ouvrir le relai 2
void R2_ouvert(void)
{
    Set_R2=1;
    __delay_ms(10);
    Set_R2=0;
}

// Fonction pour fermer le relai 2
void R2_fermée(void)
{
    Reset_R2=1;
    __delay_ms(10);
    Reset_R2=0;
}

// Fonction pour envoyer une chaîne de
caractères via UART
void envoyer_UART(char* buffer)
{
    for(short i=0; buffer[i]!='\0'; i++)
    {
        UartWriteChar(buffer[i]);
    }
}

// Envoyer une condition de démarrage I2C
void I2C_Start(void) {
    SSP1CON2bits.SEN = 1; // Générer une
condition de start p.255 bit 0 du SSP2CON2
    while (SSP1CON2bits.SEN); // Attendre
la fin de la condition de start
}

// Envoyer une condition d'arrêt I2C
void I2C_Stop(void) {
    SSP1CON2bits.PEN = 1; // Générer une
condition de stop p.255 bit 2 du SSP2CON2
    while (SSP1CON2bits.PEN); // Attendre
la fin de la condition de stop
}

// Envoyer un octet sur le bus I2C
void I2C_Write(unsigned char data) {
    SSP1BUF = data; // Charger l'octet à
envoyer
    while (SSP1STATbits.BF); //
Wait for the data to be sent

```

```

    while (SSP1CON2bits.ACKSTAT); // Check
for ACK (0 = ACK received)
//while(SSP1CON2bits.ACKSTAT==1);
//while(SSPSTATbits.RW);
//while (!PIR1bits.SSP1IF); // Attendre
que la transmission soit terminée
//PIR1bits.SSP1IF = 0; // Effacer
le flag
}
// Lire un octet depuis le bus I2C
unsigned char I2C_Read(unsigned char
ack0_nack1) {
    unsigned char data;
    SSP1CON2bits.RCEN = 1; // Activer la
réception p.255 bit 3 du SPPCON2
    while (!SSP1STATbits.BF); // Attendre
que la réception soit terminée bit0 p.252
    data = SSP1BUF; // Lire l'octet reçu
    // Envoyer un ACK ou un NACK
    SSP1CON2bits.ACKDT = ack0_nack1; // 0 =
ACK, 1 = NACK p.255 bit 5 du SPPCON2
    SSP1CON2bits.ACKEN = 1; // Envoyer le
bit d'acknowledge
    while (SSP1CON2bits.ACKEN); // Attendre
la fin de l'acknowledge
    return data;
}
// Initialisation du module I2C
void I2C_Init(void) {
    SSP1CON1 = 0x28; //0b0010 1000 p.254
//0010(bits7-4) Active le port série et
configure le SDaX et le SCL
//1000(bits3-0) I2C Host mode, clock =
FOSC / (4 * (SSPxADD+1))
    SSP1CON2 = 0; //p.255
    SSP1ADD = (_XTAL_FREQ / (4 * 100000)) -
1; // Baudrate 100 kHz P.258 formule p.251
    SSP1STAT = 0;
}
// Lire la température depuis le capteur
SHT21
float SHT21_lecture_Temperature(void) {
    unsigned int valeur=0;
    float temperature=0;

    I2C_Start();
    I2C_Write((0x40 << 1); // Adresse
SHT21 0x40 + bit0 d'écriture(0)
    I2C_Write(0xE3); // Envoyer la commande
de mesure de la température 0b1110 0011 =
0xE3
    I2C_Stop();
    I2C_Start();
    I2C_Write((0x40 << 1) | 1); // Adresse
SHT21 0x40 + bit0 de lecture (1)
    valeur = (I2C_Read(0) << 8); // Lire
le MSB avec ACK
    valeur |= I2C_Read(1); // Lire
le LSB avec NACK
    I2C_Stop();

    // Conversion des données en température
(datasheet SHT21 p.10)
    temperature = (-46.85 + 175.72 *
((float)valeur / 65536.0));

    return temperature;
}
// Lire l'humidité depuis le capteur SHT21

```

```

float SHT21_lecture_humidite(void) {
    unsigned int valeur=0;
    float humidite=0;

    I2C_Start();
    I2C_Write(0x40 << 1); // Adresse
SHT21 0x40 + bit0 d'écriture(0)
    I2C_Write(0xE5); // Envoyer la commande
de mesure de l'humidité 0b1110 0101 = 0xE5
    I2C_Stop();
    I2C_Start();
    I2C_Write((0x40 << 1) | 1); // Adresse
SHT21 0x40 + bit0 de lecture (1)
    valeur = (I2C_Read(0) << 8); // Lire
le MSB avec ACK
    valeur |= I2C_Read(1); // Lire
le LSB avec NACK
    I2C_Stop();

    // Conversion des données brutes en
humidité (datasheet SHT21 p.10)
    humidite = (-6.0 + 125.0 *
((float)valeur / 65536.0));

    return humidite;
}
//fonction pour effacer une chaîne de
caractère
void effacer(char *chaîne) {
    int i = 0;
    while (chaîne[i] != '\0') {
        chaîne[i++] = '\0';
    }
}
//configuration de l'esp(creation du server
TCP)
void Iinit_ESP(char *envoi){
    RCSTAbits.CREN = 0; // Désactive le
module réception
    sprintf(envoi,"ATE0\r\n");//commande pour
désactiver l'echo
    envoyer_UART(envoi);//désactiver echo
    __delay_ms(300);//attente qui permet à l
esp de bien prendre en compte la commande
    sprintf(envoi,"AT+CIPMUX=1\r\n");//On
autorise plusieurs connexions parallèles
    envoyer_UART(envoi);//envoi la commande a
l'ESP
    __delay_ms(300);//attente qui permet à l
esp de bien prendre en compte la commande
    sprintf(envoi,"AT+RFPower=0\r\n"); //
puissance de TX (0 ~ 82, unit:0.25dBm)
    envoyer_UART(envoi);//envoi la commande a
l'ESP
    __delay_ms(300);//attente qui permet à l
esp de bien prendre en compte la commande
    sprintf(envoi,"AT+CIPSERVER=1,8080\r\n");//(
1 : Active le serveur,8080 : PORT)
    envoyer_UART(envoi);//envoi la commande a
l'ESP
    __delay_ms(300);//attente qui permet à l
esp de bien prendre en compte la commande
    RCSTAbits.CREN = 1; // active le module
réception
    effacer(envoi); //effacer chaîne de
caractère
}
void Init_Interruption(void){

```

```

    PIE1bits.RC1IE=1; //active les
    interruptions en réception (sur RX)) p.112
    INTCONbits.TMR0IE=1; //active l
    interruption sur l ocerflow du timer0 p.109
    INTCONbits.PEIE=1; //active les
    interruptions périphérique p.109
    INTCON2bits.TMR0IP=0; //définir
    l'interruption du timer0 comme basse
    priorité p.110
    IPR1bits.RC1IP=1; //Définir réception
    UART comme haut priorité p.121
    RCONbits.IPEN=1; //Active système de
    priorités d'interruptions p.56
    INTCONbits.GIE=1; //active les
    interruptions Global p.109
}
void Init_Timer0(void) // Configuration du
Timer0 pour 0,2Hz (T=5s)
{
    T0CONbits.TMR0ON=0; //timer désactivé p.154
    T0CONbits.T08BIT=0; //configuration du
    compteur sur 16bits p.154
    T0CONbits.T0SE=0; // incrémentation sur
    front montant p.154
    T0CONbits.T0CS=0; //Fosc/4 p.154
    T0CONbits.PSA=0; //Activation du
    prédiviseur P.154
    T0CONbits.T0PS=4; //prédiviseur par 32 :
    Fos/(4*32)=1M/128= 7812,5Hz->T=128µs p.154
    // Calcul du préchargement : 65536 -
    (39062) = 26474
    // En hexadécimal : 26474 = 0x676A
    TMR0H= 0x67; // Partie haute du registre
    TMR0 (0x67)
    TMR0L = 0x6A; // Partie basse du registre
    TMR0 (0x6A)
    INTCONbits.TMR0IF = 0; // RAZ du flag
    TMR0IF
}
//-----
//Initialisation générale du Pic
void InitPic(void)
{
    /*
    //Initialisation du vecteur d'interruption
    #asm
    PSECT VectInterrupt,class=CODE,abs
    ORG 008h
        goto _TraitementIT
    ORG 018h
        goto _TraitementIT
    #endasm
    */
    //Au
    reset INTSRC=0 et MFIOSEL=0

    #if FCLK==1
    //Tcyc=4µs
    OSCCON=0b00110010;
    //IRCF=%011 (1 MHz) et SCS=%10: horloge
    interne réglée sur 1 MHz
    #endif

    #if FCLK==2
    //Tcyc=2µs

```

```

    OSCCON=0b01000010;
    //IRCF=%100 (2 MHz) et SCS=%10: horloge
    interne réglée sur 2 MHz
    #endif

    #if FCLK==4
    //Tcyc=1µs
    OSCCON=0b01010010;
    //IRCF=%101 (4 MHz) et SCS=%10: horloge
    interne réglée sur 4 MHz
    #endif

    #if FCLK==8
    //Tcyc=500ns
    OSCCON=0b01100010;
    //IRCF=%110 (8 MHz) et SCS=%10: horloge
    interne réglée sur 8 MHz
    #endif

    #if FCLK==16
    //Tcyc=250ns
    OSCCON=0b01110010;
    //IRCF=%111 (16 MHz) et SCS=%10: horloge
    interne réglée sur 16 MHz
    #endif

    #if FCLK==20
    //Tcyc=200ns
    OSCCON=0b00000000;
    //IRCF=%000 horloge interne réglée sur 31kHz
    dévalidée
    #endif

    #if FCLK==32
    //Tcyc=125ns
    OSCCON=0b01100000;
    //IRCF=%110 (8 MHz) et SCS=%00: horloge
    interne réglée sur 8 MHz
    OSTUNEbits.PLEN=1; //active
    la PLL
    #endif

    #if FCLK==64
    //Tcyc=62.5ns
    OSCCON=0b01110000;
    //IRCF=%111 (16 MHz) et SCS=%00: horloge
    interne réglée sur 16 MHz
    OSTUNEbits.PLEN=1; //active
    la PLL
    #endif

    // INTCONbits.PEIE=1;
    //validation des interruptions périphériques

    STATM=0;

    //Initialisation des ports d'entrée/sortie

    //RB0 (Reset_R1), RB1 (Set_R1), RB2
    (Reset_R2), RB3 (Set_R2) en sortie logique
    numérique
    TRISBbits.RB0=0; //RB0 en sortie
    TRISBbits.RB1=0; //RB1 en sortie
    TRISBbits.RB2=0; //RB2 en sortie
    TRISBbits.RB3=0; //RB3 en sortie
    ANSELBbits.ANSB0=0; //désactivation du mode
    analogique de RB0
    ANSELBbits.ANSB1=0; //désactivation du mode
    analogique de RB1

```

```

    ANSELBbits.ANSB2=0;//désactivation du mode
    analogique de RB2
    ANSELBbits.ANSB3=0;//désactivation du mode
    analogique de RB3
    // Configuration des broches I2C
    TRISCbits.RC3 = 1; // Configurer RC3
    (SCL) en entrée
    TRISCbits.RC4 = 1; // Configurer RC4
    (SDA) en entrée
    ANSELCbits.ANSC3=0; //désactivation du
    mode analogique de RB0
    ANSELCbits.ANSC4=0; //désactivation du
    mode analogique de RB1

    Reset_R1=0;
    Set_R1=0;
    Reset_R2=0;
    Set_R2=0;

    TEF_bit=0;
    VaON_bit=0;

}
unsigned char envoi[30];
char data[20];
char recu[20];
char capteur[30];
char connexion=0;

void main (void)
{
    InitPic(); // Configuration du PIC
    I2C_Init(); // configuration du module de
    communication I2C
    InitUart9600(FCLK); // Initialisation de
    la communication UART à 9600 bauds
    Init_Interruption(); // Configuration des
    interruptions
    Init_Timer0(); // Configuration du Timer0
    pour envoi des données toutes les 5s
    Init_ESP(envoi); // Initialisation du
    module ESP pour la communication WiFi
    BAUDCONbits.WUE=1; //Active le réveille
    sur RCIF p.127
    float temperature=0;
    float humidite=0;
    while (1) // Boucle infinie
    {
        if(connexion==0) // Si aucune
        connexion n'est établie
        {
            SLEEP(); // Met le microcontrôleur
            en mode veille pour économiser l'énergie
        }
    }
}
// Routine d'interruption de haute priorité
void __interrupt (high_priority) HighISR() {
    char raz; // Variable de
    réinitialisation
    static unsigned short i=0; // Index du
    buffer de réception
    char caractere_recu; // Stocke le
    caractère reçu
    unsigned short etat_relai_1; // État du
    relais 1
    unsigned short etat_relai_2; // État du
    relais 2

```

```

        if (PIR1bits.RCIF == 1) { // Vérifie si
        une interruption de réception UART est
        déclenchée
            // Resynchronisation de l'UART
            (perte du premier octet du au temps de
            reveille par la reception UART)
            RCSTAbits.CREN = 0; // Désactive la
            réception UART
            __delay_ms(1); // Petit délai pour
            stabiliser la réception
            raz = RCREG1; // Lecture du registre
            de réception pour vider le buffer
            RCSTAbits.CREN = 1; // Réactivation
            de la réception UART

            do {
                caractere_recu = UartReadChar();
            // Lire un caractère depuis l'UART
                data[i] = caractere_recu; //
            Stocker dans le buffer de réception
                i++;
            } while(((caractere_recu != '\n' )||
            i <= 2) && (i <= 11)); // Lire jusqu'à '\n'
            ou atteindre la taille max

            i=0; // Réinitialisation de l'index

            if(strncmp(data+2,"IPD,0,2:", 8) ==
            0){ // Vérifie si la commande reçue est une
            commande de mise à jour des relais
                etat_relai_1 = (data[10] - '0');
            // Conversion du caractère en valeur
            numérique (0 ou 1)
                etat_relai_2 = (data[11] - '0');

                if(etat_relai_1){
                    R1_ouvert(); // Ouvre le
                    relais 1
                } else {
                    R1_fermee(); // Ferme le
                    relais 1
                }

                if(etat_relai_2){
                    R2_ouvert(); // Ouvre le
                    relais 2
                } else {
                    R2_fermee(); // Ferme le
                    relais 2
                }
            }
            else if(strncmp(data+1,"CONNECT", 7)
            == 0){ // Vérifie si une connexion est
            établie
                T0CONbits.TMR0ON=1; //timer
                activé p.154
                TMR0H= 0x67; // Partie haute du
                registre TMR0 (0x67)
                TMR0L = 0x6A; // Partie basse du
                registre TMR0 (0x6A)
                INTCONbits.TMR0IF = 0; // RAZ
                du flag TMR0IF
                connexion=1;
            }
            else if(strncmp(data+1,"CLOSED", 6)
            == 0){ // Vérifie si la connexion est fermée
                T0CONbits.TMR0ON = 0; //
                Désactive le Timer0

```

```

        effacer(data); // Efface les
données reçues
        connexion = 0; // Marque la
connexion comme inactive
    }
}
BAUDCONbits.WUE=1; //Active le réveille
sur RCIF p.127
}
void __interrupt (low_priority) LowISR() {
    float temperature=0;
    float humidite=0;
    char raz;
    if (INTCONbits.TMR0IF){
        // Réinitialiser le Timer0 pour la
prochaine période de comptage
        TMR0H= 0x67; // Partie haute du
registre TMR0 (0x67)
        TMR0L = 0x6A; // Partie basse du
registre TMR0 (0x6A)
        INTCONbits.TMR0IF = 0; // RAZ du
flag TMR0IF
    }
}

```

```

        //mesure de la température et de
l'humidité + envoi
        humidite=SHT21_lecture_humidite();
//mesure humidité

temperature=SHT21_lecture_Temperature();
//mesure température

sprintf(capteur,"%0.1f%0.1f",temperature,humi
dite);
    sprintf(envoi,"AT+CIPSEND=0,9\r\n");
    envoyer_UART(envoi); // Envoie la
commande AT pour transmettre les données
    __delay_ms(100); // Attente pour
stabiliser l'envoi
    envoyer_UART(capteur); // Envoie les
données de température et humidité
    }
}

```

13.4. Codes node.js, HTML et CSS

```

// Server.js

const express = require('express');
const http = require('http');
const socketIo = require('socket.io');

const path=require('path')

// Utiliser le wifi
const wifi = require('node-wifi');

// Variables globales
var global_ssid = {}; //crée une variable
globale pour le ssid

// Crée une instance d'Express et un
serveur HTTP
const app = express();
const server = http.createServer(app);
const io = socketIo(server);

// Sert à utiliser les fichiers statiques
(HTML, CSS, JS) depuis le dossier 'public'
app.use(express.static('public'));
app.get('/', (req, res) => {
    res.sendFile(path.join(__dirname,
'public', 'home.html'));
});

// Écouter les connexions des clients via
Socket.IO
io.on('connection', (socket) => {
    console.log('Un client est
connecté. ');

    //---IP---
    // Écouter l'événement 'send-ssid'
envoyé depuis le client
    socket.on('send-ssid', (ssid) => {

```

```

        console.log(`IP reçue du client:
${ssid}`);
        global_ssid.ssid=ssid
        // Répondre au client avec le ssid
reçue
        socket.emit('receive-ssid', `Vous
avez envoyé le ssid : ${ssid}`);
    });

    //---MDP---
    // Écouter l'événement 'getmdp' envoyé
depuis le client
    socket.on('getmdp', (mdp) => {
        console.log(`mdp reçu du client:
${mdp}`);
        wifi.connect({ ssid:
global_ssid.ssid , password: mdp }, () => {
            wifi.getCurrentConnections((error,
currentConnections) => { // Liste les
connections wifi actuelles
                if (error) {
                    console.log(error);
                } else {
                    console.log(currentConnections
);
                }
            });
        });
        socket.emit('receive-
mdp', `connection en cours`);
    });

    // Événement de déconnexion
    socket.on('disconnect', () => {
        console.log('Un client a
quitté. ');
    });
});

```

```
// Démarrer le serveur sur le port 3000
server.listen(3000, () => {
  console.log('Serveur démarré sur
http://localhost:3000');
});

////-----WIFI-----

wifi.init({
  iface: null, // Utilise l'interface
par défaut
});
app.get('/scan', async (req, res) => {
  try {
    const networks = await wifi.scan();
    res.json(networks);
  } catch (error) {
    res.status(500).json({ error:
'Erreur lors du scan des Wi-Fi.' });
  }
});

// Client.js

const socket = io(); // Connexion au
serveur Socket.IO

let ssidSelectionne = false; // Variable
pour vérifier si un SSID est sélectionné
ou envoyé

//---SSID envoyé manuellement---
function envoysssid() {
  const ssid =
document.getElementById("ssidInput").value
;
  if (ssid) {
    socket.emit('send-ssid', ssid); //
Envoi du SSID au serveur
    ssidSelectionne = true; // Marquer
qu'un SSID a été envoyé
    document.getElementById('resultat'
).textContent = `Réseau sélectionné :
${ssid}`;
    afficherBoutonSuivant(); //
Vérifier pour afficher le bouton/liens
  } else {
    alert("Veuillez entrer un SSID
valide.");
  }
}

//---Scan Wi-Fi---
function scannerWifi() {
  fetch('/scan') // Appelle l'API
'/scan' du serveur
  .then(response => response.json())
// Récupère la liste des réseaux au format
JSON
  .then(networks => {
    const wifiList =
document.getElementById('wifiList'); //
Cible la liste des réseaux
    wifiList.innerHTML = ''; //
Réinitialise la liste des réseaux
```

```
// Vérifie s'il y a des
réseaux disponibles
if (networks.length === 0) {
  wifiList.innerHTML =
'<p>Aucun réseau Wi-Fi trouvé.</p>';
  return;
}

// Ajoute chaque réseau à la
liste
networks.forEach(network => {
  const item =
document.createElement('div');
  item.className = 'wifi-
item';
  item.innerHTML = `
    <span>${network.ssid
|| 'Réseau masqué'}</span>
  `;

  // Ajouter un événement de
clic pour sélectionner un réseau
  item.addEventListener('cli
ck', () => {
    const ssid =
network.ssid || 'Réseau masqué';
    document.getElementById('resultat').textContent = `Réseau
sélectionné : ${ssid}`;
    socket.emit('send-
ssid', ssid); // Envoi du SSID au serveur
    ssidSelectionne =
true; // Marquer qu'un SSID a été
sélectionné
    afficherBoutonSuivant(
); // Vérifier pour afficher le
bouton/liens
  });

  wifiList.appendChild(item)
;
});
}.catch(error => {
  console.error('Erreur lors du
scan des réseaux Wi-Fi :', error);
  document.getElementById('wifiL
ist').innerHTML = '<p>Erreur lors du scan
des réseaux.</p>';
});
}

//---Affichage conditionnel du
bouton/liens---
function afficherBoutonSuivant() {
  if (ssidSelectionne) {
    document.getElementById('nextPageB
utton').style.display = 'inline-block'; //
Affiche le bouton/liens
  }
}

//---Mot de passe---
function connection() {
```

```

    const mdp =
document.getElementById("mdp").value; //
Récupère le mot de passe saisi

    if (!mdp) {
        alert("Veuillez entrer un mot de
passe.");
        return;
    }

    afficherOverlay("Connexion en
cours..."); // Afficher l'overlay avec un
message

    // Envoi du mot de passe au serveur
socket.emit('getmdp', mdp);

    // Écoute la réponse du serveur
socket.on('receive-mdp', (message) =>
{
    if (message === "success") {
        afficherOverlay("Connexion
réussie ! Redirection...");
        setTimeout(() => {
            window.location.href =
"page_ui_carte.html"; // Redirige vers
l'interface principale
        }, 2000); // Attendre 2
secondes avant la redirection
    } else {
        afficherOverlay("Connexion
échouée. Veuillez réessayer.");
        setTimeout(() => {
            cacherOverlay(); //
Masquer l'overlay après un échec
        }, 2000);
    }
});

// Fonction pour afficher l'overlay avec
un message
function afficherOverlay(message) {
    const overlay =
document.getElementById("overlay");
    const overlayMessage =
document.getElementById("overlay-
message");

    overlayMessage.textContent = message;
    overlay.style.display = "flex"; //
Affiche l'overlay
}

// Fonction pour masquer l'overlay
function cacherOverlay() {
    const overlay =
document.getElementById("overlay");
    overlay.style.display = "none"; //
Masque l'overlay
}

// Fonction pour retourner a la page
principale
function retour() {

```

```

window.location.href = "home.html"; //
Redirige vers la page d'accueil
}

```

13.4.1. HTML

```

13.4.1.1. home.html

<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
    <title>Projet carte Wi-Fi</title>
    <link rel="icon" type="image/x-icon"
href="ICOCW.ico"> <!-- Icône -->
    <link rel="stylesheet"
href="styles.css"> <!-- Lien vers le
fichier CSS -->
    <script
src="/socket.io/socket.io.js"></script>
<!-- Script Socket.IO -->
    <script src="client.js"></script> <!--
Script client.js -->
</head>

<body>
    <h1>Sélection du Wi-Fi</h1>
    <h2>Entrez un SSID ou sélectionnez un
réseau Wi-Fi</h2>

    <label for="ipInput">SSID :</label>
    <input type="text" id="ssidInput"
placeholder="Ex:nom_du_wifi">
    <button
onclick="envoyerssid()">Envoyer</button>

    <p></p>
    <button
onclick="scannerWifi()">Scanner les
réseaux</button>
    <div id="wifiList" class="wifi-list">
        <p>Aucun réseau trouvé pour le
moment...</p>
    </div>

    <p id="resultat"></p>

    <!-- Bouton/liens vers la page suivante
(masqué par défaut) -->
    <a id="nextPageButton"
href="page_connection.html"
style="display: none;">Passer à la page de
connexion</a>
</body>
</html>

```

```

13.4.1.2. page_connection.html

<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">

```

```

    <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
    <title>Projet carte Wi-Fi</title>
    <link rel="icon" type="image/x-icon"
href="ICOCW.ico"> <!-- Icône -->
    <link rel="stylesheet"
href="styles.css"> <!-- Lien vers le
fichier CSS -->
    <script
src="/socket.io/socket.io.js"></script>
<!-- Script Socket.IO -->
    <script src="client.js"></script> <!--
Script client.js -->
</head>
<body>
    <h1>Page de connexion</h1>
    <h2>Entrez un mot de passe</h2>
    <label for="mdp">Mot de passe
:</label>
    <input type="text" id="mdp"
placeholder="Mot de passe">
    <button onclick="connection()">Se
connecter</button>
    <p id="connection"></p>

    <!-- Bouton de retour -->
    <button
onclick="retour()">Retour</button>

    <!-- Overlay pour afficher les
messages dynamiques -->
    <div id="overlay" style="display:
none;">
        <div id="overlay-content">
            <p id="overlay-
message">Connexion en cours...</p>
        </div>
    </div>
</body>
</html>

```

13.4.1.3. page_ui_carte.html

```

<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
    <title>Projet carte Wi-Fi</title>
    <link rel="icon" type="image/x-icon"
href="ICOCW.ico"> <!-- Icône -->
    <link rel="stylesheet"
href="styles.css"> <!-- Lien vers le
fichier CSS -->
    <script
src="/socket.io/socket.io.js"></script>
<!-- Script Socket.IO -->
    <script src="client.js"></script> <!--
Script client.js -->
</head>
<body>
    <h1>Page de la carte</h1>
</body>

```

```
</html>
```

13.4.2. CSS

13.4.2.1. styles.css

```

/* Styles globaux */
body {
    font-family: Arial, sans-serif;
    background-color: #FEFCEA;
    color: #333;
    text-align: center;
    padding: 20px;
}

/* Titres */
h1, h2 {
    color: #003D4B;
}

/* Boutons et liens en forme de boutons */
button, a#nextPageButton {
    padding: 10px 20px;
    background-color: #003D4B;
    color: white;
    text-decoration: none;
    border-radius: 5px;
    font-size: 16px;
    font-weight: bold;
    text-align: center;
    box-shadow: 0 4px 6px rgba(0, 0, 0,
0.1);
    cursor: pointer;
    margin-top: 10px;
}

button:hover, a#nextPageButton:hover {
    background-color: #00506E;
}

/* Lien vers la page de connexion (caché
par défaut) */
a#nextPageButton {
    display: none; /* Caché par défaut */
}

/* Liste des réseaux Wi-Fi */
.wifi-list {
    margin: 20px auto;
    max-width: 500px;
    background: white;
    border-radius: 10px;
    border: 1px solid #C2C0B3;
    box-shadow: 0 4px 6px rgba(0, 0, 0,
0.1);
    padding: 20px;
}

.wifi-item {
    display: flex;
    justify-content: space-between;
    padding: 10px;
    border-bottom: 1px solid #ddd;
}

```

```
.wifi-item:hover {
    background-color: #f0f4f9;
    cursor: pointer;
}

.wifi-item:last-child {
    border-bottom: none;
}

/* Overlay pour l'état de la connexion */
#overlay {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(0, 0, 0, 0.5);
    /* Fond semi-transparent */
    display: flex;
    justify-content: center;
    align-items: center;
    z-index: 1000;
}

#overlay-content {
    background-color: white;
    padding: 20px;
    border-radius: 10px;
    text-align: center;
    box-shadow: 0 4px 6px rgba(0, 0, 0,
0.1);
}

#overlay-message {
    font-size: 18px;
    font-weight: bold;
    color: #333;
}
```